

Objectives

- To demonstrate the working of process scheduling algorithms.

Pre-Lab Theory:

First come First serve algorithm:

```
#include<stdio.h>
```

```
main()
{
    int bt[20], wt[20], tat[20], i, n; float
    wtsum, tatsum;
    //clrscr();
    printf("\nEnter the number of processes -- "); scanf("%d",
    &n);
    for(i=0;i<n;i++)
    {
        printf("\nEnter Burst Time for Process %d -- ", i); scanf("%d",
        &bt[i]);
    }
    wt[0] = wtavg = 0; tat[0]
    = tatavg = bt[0];
    for(i=1;i<n;i++)
    {
        wt[i] = wt[i-1] +bt[i-1];
        tat[i] = tat[i-1] +bt[i];
        wtsum = wtsum + wt[i]; tatsum
        = tatsum + tat[i];
    }
    printf("\t PROCESS \tBURST TIME \t WAITING TIME\t TURNAROUND
    TIME\n");
    for(i=0;i<n;i++)
    printf("\n\t P%d \t\t %d \t\t\t %d \t\t\t %d", i, bt[i], wt[i],
    tat[i]);
    printf("\nAverage Waiting Time -- %f", wtsum/n);printf("\n
    Average Turnaround Time -- %f", tatsum/n);}
```

In-Lab Tasks:

Task1

Add arrival times to the FCFS Algorithm code given in the prelab to implement the complete FCFS algorithm in c language.

```
#include <stdio.h>

int main() {
    int n, i;
    int at[20], bt[20], wt[20], tat[20];
    float wsum = 0, tasum = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time for P%d: ", i);
        scanf("%d", &at[i]);
        printf("Enter Burst Time for P%d: ", i);
        scanf("%d", &bt[i]);
    }

    // FCFS works on arrival order → sort by arrival time
    for(int k = 0; k < n-1; k++){
        for(int j = 0; j < n-k-1; j++){
            if(at[j] > at[j+1]){
                int t = at[j]; at[j] = at[j+1]; at[j+1] = t;
                t = bt[j]; bt[j] = bt[j+1]; bt[j+1] = t;
            }
        }
    }

    wt[0] = 0;
    tat[0] = bt[0];

    for(i = 1; i < n; i++){
        if(tat[i-1] < at[i])
            wt[i] = 0; // CPU idle until process arrives
        else
            wt[i] = tat[i-1] - at[i];

        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tAT\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], wt[i], tat[i]);
        wsum += wt[i];

        tasum += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", wsum/n);
    printf("\nAverage Turnaround Time = %.2f\n", tasum/n);

    return 0;
}
```

Brief the working/output:

```
muhammad-ahmad@latitude-7490:~/os/lab8$ ./task.o
```

Process	AT	BT	WT	TAT
P0	0	4	0	4
P1	5	3	0	3
P2	6	2	0	2
P3	12	5	0	5

Average Waiting Time = 0.00

Average Turnaround Time = 3.50

Task2

Implement Shortest Job First algorithm in c programming language.

```
#include <stdio.h>

int main() {
    int n, i, j, completed = 0, t = 0, min, p;
    int at[20], bt[20], done[20], wt[20], tat[20];
    float wsum = 0, tasum = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i=0; i<n; i++){
        printf("Enter Arrival Time for P%d: ", i);
        scanf("%d", &at[i]);
        printf("Enter Burst Time for P%d: ", i);
        scanf("%d", &bt[i]);
        done[i] = 0;
    }

    while(completed < n){
        min = 9999;
        p = -1;

        for(i=0; i<n; i++){
            if(at[i] <= t && !done[i] && bt[i] < min){
                min = bt[i];
                p = i;
            }
        }

        if(p == -1){
            t++;
            continue;
        }
        wt[p] = t - at[p];
        t += bt[p];
        tat[p] = t - at[p];
        done[p] = 1;
        completed++;
    }
}
```

```
printf("\nProcess\tAT\tBT\tWT\tTAT\n");
for(i=0; i<n; i++){
    printf("P%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], wt[i], tat[i]);
    wsum += wt[i];
    tasum += tat[i];
}

printf("\nAverage Waiting Time = %.2f", wsum/n);
printf("\nAverage Turnaround Time = %.2f\n", tasum/n);

return 0;
}
```

Brief the working/output:

muhammad-ahmad@latitude-7490:~/os/lab8\$./task.o

<i>Process</i>	<i>AT</i>	<i>BT</i>	<i>WT</i>	<i>TAT</i>
<i>P0 0</i>	<i>4</i>	<i>0</i>	<i>4</i>	
<i>P1 5</i>	<i>3</i>	<i>0</i>	<i>3</i>	
<i>P2 6</i>	<i>2</i>	<i>2</i>	<i>4</i>	
<i>P3 12</i>	<i>5</i>	<i>0</i>	<i>5</i>	

Average Waiting Time = 0.50

Average Turnaround Time = 4.00

Task3

Implement the Shortest Remaining Time First algorithm in c programming language.

```
#include <stdio.h>

int main(){
    int n, i, t = 0, completed = 0, minm;
    int at[20], bt[20], rt[20], wt[20], tat[20];
    float wsum = 0, tasum = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i=0;i<n;i++){
        printf("Enter Arrival Time for P%d: ", i);
        scanf("%d", &at[i]);
        printf("Enter Burst Time for P%d: ", i);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    while(completed < n){
        int shortest = -1;
        minm = 9999;

        for(i=0;i<n;i++){
            if(at[i] <= t && rt[i] > 0 && rt[i] < minm){
                minm = rt[i];
                shortest = i;
            }
        }
        if(shortest == -1){
            t++;
            continue;
        }
        rt[shortest]--;
        t++;

        if(rt[shortest] == 0){
            completed++;
            tat[shortest] = t - at[shortest];
            wt[shortest] = tat[shortest] - bt[shortest];
        }
    }

    printf("\nProcess\tAT\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], wt[i], tat[i]);
        wsum += wt[i];
        tasum += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", wsum/n);
    printf("\nAverage Turnaround Time = %.2f\n", tasum/n);

    return 0;
}
```

Brief the working/output:

```
muhammad-ahmad@latitude-7490:~/os/lab8$ ./task.o
```

<i>Process</i>	<i>AT</i>	<i>BT</i>	<i>WT</i>	<i>TAT</i>
<i>P0</i>	<i>0</i>	<i>4</i>	<i>0</i>	<i>4</i>
<i>P1</i>	<i>5</i>	<i>0</i>	<i>3</i>	<i>3</i>
<i>P2</i>	<i>6</i>	<i>2</i>	<i>2</i>	<i>4</i>
<i>P3</i>	<i>12</i>	<i>5</i>	<i>0</i>	<i>5</i>

Average Waiting Time = 0.50

Average Turnaround Time = 4.00

*Task4**Implement the Round Robin algorithm in c programming language*

```
#include <stdio.h>
int main(){
    int n, tq, i, t = 0, done = 0;
    int at[20], bt[20], rt[20], wt[20], tat[20];
    float wsum = 0, tasum = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
    for(i=0;i<n;i++){
        printf("Enter Arrival Time for P%d: ", i);
        scanf("%d", &at[i]);
        printf("Enter Burst Time for P%d: ", i);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    while(done < n){
        int flag = 0;
        for(i=0;i<n;i++){
            if(at[i] <= t && rt[i] > 0){
                flag = 1;
                if(rt[i] > tq){
                    rt[i] -= tq;
                    t += tq;
                }
                else{
                    t += rt[i];
                    tat[i] = t - at[i];
                    wt[i] = tat[i] - bt[i];
                    rt[i] = 0;
                    done++;
                }
            }
        }

        if(!flag) t++; // CPU idle
    }
    printf("\nProcess\tAT\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], wt[i], tat[i]);
        wsum += wt[i];
        tasum += tat[i];
    }
    printf("\nAverage Waiting Time = %.2f", wsum/n);
    printf("\nAverage Turnaround Time = %.2f\n", tasum/n);

    return 0;
}
```

Brief the working/output:

muhammad-ahmad@latitude-7490:~/os/lab8\$./task.o

Process	AT	BT	WT	TAT
P0	0	4	0	4
P1	5	3	2	5
P2	6	2	1	3
P3	12	5	0	5

Average Waiting Time = 0.75

Average Turnaround Time = 4.25

Task5 :Compare all algorithms.

```
#include <stdio.h>

#define MAX 20

/* ----- FCFS ----- */
void fcfs(int n, int at[], int bt[], float *avgWT, float *avgTAT) {
    int wt[MAX], tat[MAX];
    float wsum = 0, tasum = 0;

    wt[0] = 0;
    tat[0] = bt[0];

    for(int i = 1; i < n; i++){
        if(tat[i-1] < at[i])
            wt[i] = 0;
        else
            wt[i] = tat[i-1] - at[i];

        tat[i] = wt[i] + bt[i];
    }

    for(int i = 0; i < n; i++){
        wsum += wt[i];
        tasum += tat[i];
    }

    *avgWT = wsum / n;
    *avgTAT = tasum / n;

    printf("\n--- FCFS ---\n");
    printf("Process\tAT\tBT\tWT\tTAT\n");
    for(int i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], wt[i], tat[i]);
}

/* ----- SJF (Non-preemptive) ----- */
void sjf(int n, int at[], int bt[], float *avgWT, float *avgTAT) {
    int rt[MAX], wt[MAX], tat[MAX], done[MAX];
    float wsum = 0, tasum = 0;
    for(int i=0;i<n;i++){ rt[i] = bt[i]; done[i] = 0; }
```

LAB # 8 process scheduling algorithms

```
int completed = 0, t = 0;
while(completed < n){
    int shortest = -1, min = 9999;

    for(int i=0;i<n;i++){
        if(at[i] <= t && !done[i] && bt[i] < min){
            min = bt[i];
            shortest = i;
        }
    }

    if(shortest == -1){ t++; continue; }

    wt[shortest] = t - at[shortest];
    t += bt[shortest];
    tat[shortest] = t - at[shortest];
    done[shortest] = 1;
    completed++;
}

for(int i=0;i<n;i++){
    wsum += wt[i];
    tasum += tat[i];
}

*avgWT = wsum / n;
*avgTAT = tasum / n;

printf("\n--- SJF (Non-preemptive) ---\n");
printf("Process\tAT\tBT\tWT\tTAT\n");
for(int i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], wt[i], tat[i]);
}

/* ----- SRTF (Preemptive SJF) ----- */
void srtf(int n, int at[], int bt[], float *avgWT, float *avgTAT) {
    int rt[MAX], wt[MAX], tat[MAX];
    float wsum = 0, tasum = 0;

    for(int i=0;i<n;i++)
        rt[i] = bt[i];

    int completed = 0, t = 0;

    while(completed < n){
        int shortest = -1, min = 9999;

        for(int i=0;i<n;i++){
            if(at[i] <= t && rt[i] > 0 && rt[i] < min){
                min = rt[i];
                shortest = i;
            }
        }

        if(shortest == -1){ t++; continue; }

        rt[shortest]--;
        t++;

        if(rt[shortest] == 0){
            completed++;
        }
    }
}
```

```

        tat[shortest] = t - at[shortest];
        wt[shortest] = tat[shortest] - bt[shortest];
    }
}

for(int i=0;i<n;i++){
    wsum += wt[i];
    tasum += tat[i];
}

*avgWT = wsum / n;
*avgTAT = tasum / n;

printf("\n--- SRTF (Preemptive SJF) ---\n");
printf("Process\tAT\tBT\tWT\tTAT\n");
for(int i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], wt[i], tat[i]);
}

/* ----- Round Robin ----- */
void rr(int n, int at[], int bt[], int tq, float *avgWT, float *avgTAT) {
    int rt[MAX], wt[MAX], tat[MAX];
    float wsum = 0, tasum = 0;
    for(int i=0;i<n;i++) rt[i] = bt[i];

    int done = 0, t = 0;

    while(done < n){
        int flag = 0;

        for(int i=0;i<n;i++){
            if(at[i] <= t && rt[i] > 0){
                flag = 1;
                if(rt[i] > tq){
                    rt[i] -= tq;
                    t += tq;
                }
                else{
                    t += rt[i];
                    tat[i] = t - at[i];
                    wt[i] = tat[i] - bt[i];
                    rt[i] = 0;
                    done++;
                }
            }
        }

        if(!flag) t++;
    }

    for(int i=0;i<n;i++){
        wsum += wt[i];
        tasum += tat[i];
    }

    *avgWT = wsum / n;
    *avgTAT = tasum / n;

    printf("\n--- Round Robin ---\n");
    printf("Process\tAT\tBT\tWT\tTAT\n");
    for(int i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], wt[i], tat[i]);
    }
}

```

LAB # 8 process scheduling algorithms

```
}

/* ----- MAIN PROGRAM ----- */
int main() {
    int n, tq;
    int at[MAX], bt[MAX];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i=0;i<n;i++){
        printf("Enter Arrival Time for P%d: ", i);
        scanf("%d", &at[i]);
        printf("Enter Burst Time for P%d: ", i);
        scanf("%d", &bt[i]);
    }

    printf("Enter Time Quantum (RR): ");
    scanf("%d", &tq);

    float fcfsWT, fcfsTAT, sjfWT, sjfTAT, srtfWT, srtfTAT, rrWT, rrTAT;

    fcfs(n, at, bt, &fcfsWT, &fcfsTAT);
    sjf(n, at, bt, &sjfWT, &sjfTAT);
    srtf(n, at, bt, &srtfWT, &srtfTAT);
    rr(n, at, bt, tq, &rrWT, &rrTAT);

    printf("\n\n===== COMPARISON TABLE =====\n");
    printf("Algorithm\tAvg WT\t\tAvg TAT\n");
    printf("FCFS\t\t%.2f\t\t%.2f\n", fcfsWT, fcfsTAT);
    printf("SJF\t\t%.2f\t\t%.2f\n", sjfWT, sjfTAT);
    printf("SRTF\t\t%.2f\t\t%.2f\n", srtfWT, srtfTAT);
    printf("Round Robin\t%.2f\t\t%.2f\n", rrWT, rrTAT);

    printf("\nBest Scheduling Algorithm Based on WAITING TIME: ");
    if(srtfWT <= sjfWT && srtfWT <= fcfsWT && srtfWT <= rrWT)
        printf("SRTF\n");
    else if(sjfWT <= fcfsWT && sjfWT <= rrWT)
        printf("SJF\n");
    else if(rrWT <= fcfsWT)
        printf("Round Robin\n");
    else
        printf("FCFS\n");

    return 0;
}
```

muhammad-ahmad@latitude-7490:~/os/lab8\$./task.o

```
Enter number of processes: 4
Enter Arrival Time for P0: 0
Enter Burst Time for P0: 4
Enter Arrival Time for P1: 5
Enter Burst Time for P1: 3
Enter Arrival Time for P2: 6
Enter Burst Time for P2: 2
Enter Arrival Time for P3: 12
Enter Burst Time for P3: 5
Enter Time Quantum (RR): 3
```

--- FCFS ---

Process	AT	BT	WT	TAT
P0	0	4	0	4

LAB # 8 process scheduling algorithms

```
P1    5    3    0    3
P2    6    2    0    2
P3   12    5    0    5
```

--- SJF (Non-preemptive) ---

```
Process    AT    BT    WT    TAT
P0         0     4     0     4
P1         5     3     0     3
P2         6     2     2     4
P3        12     5     0     5
```

--- SRTF (Preemptive SJF) ---

```
Process    AT    BT    WT    TAT
P0         0     4     0     4
P1         5     3     0     3
P2         6     2     2     4
P3        12     5     0     5
```

--- Round Robin ---

```
Process    AT    BT    WT    TAT
P0         0     4     0     4
P1         5     3     0     3
P2         6     2     2     4
P3        12     5     0     5
```

===== COMPARISON TABLE =====

Algorithm	Avg WT	Avg TAT
FCFS	0.00	3.50
SJF	0.50	4.00
SRTF	0.50	4.00
Round Robin	0.50	4.00

Best Scheduling Algorithm Based on WAITING TIME: FCFS

Table 1: Revise statements of LAB rubric for assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	